

课程讲义

从 PyTorch 算子到底层 GPU Kernel

Stanford CS336 Spring 2025 · Lecture 6 · Kernels, Triton



频道: Stanford Online

发布日期: 2025 年 5 月 2 日

视频时长: 1 小时 20 分 22 秒

视频链接: <https://www.youtube.com/watch?v=E8Mju53VB00>

整理说明: 依据完整字幕、原始代码画面和性能时间线重构。讲义把“实现正确”“测量可信”“瓶颈可解释”“编译结果可检查”视作同一个 kernel 工程闭环。

整理日期: 2026 年 7 月 29 日

目录

1	从高级框架到底层 kernel	3
1.1	五条实现路线	3
1.2	本章小结	4
2	硬件执行与并行度	5
2.1	SM、线程块和尾波	5
2.2	本章小结	7
3	可信基准测试	7
3.1	异步执行为什么会骗过计时器	7
3.2	本章小结	11
4	从表格到时间线: Profiler 推理	12
4.1	逐元素算子为什么慢	12
4.2	本章小结	14
5	Nsight 与底层瓶颈	15
5.1	从 kernel 时间进入硬件计数器	15
5.2	本章小结	17
6	CUDA 扩展: 控制权与工程成本	17
6.1	Python、C++ glue 与 <<<grid, block>>>	17
6.2	本章小结	20
7	Triton: 块级 SPMD 编程	20
7.1	程序实例而不是单线程	20
7.2	本章小结	23
8	Softmax: 归约 kernel 的完整案例	23
8.1	行级分块与稳定归一化	23
8.2	本章小结	25
9	Kernel 工程复盘: 从作业到生产	25
9.1	建立语义契约	25
9.2	设计 benchmark 套件	26
9.3	Profiler 决策树	26
9.4	CUDA 路线的维护边界	26

9.5	Triton 路线的调优边界	27
9.6	Softmax 的完整检查	27
9.7	本章小结	27

1 从高级框架到底层 kernel

1.1 五条实现路线

一个 GELU、softmax 或矩阵乘，在数学上只是一个函数，在系统里却可以有多条实现路径：直接组合 PyTorch 算子、使用 `torch.compile`、写 Triton、写 CUDA 扩展，或调用厂商库。路线越低层，控制权越多，同时正确性、边界、调度和构建负担也越大。

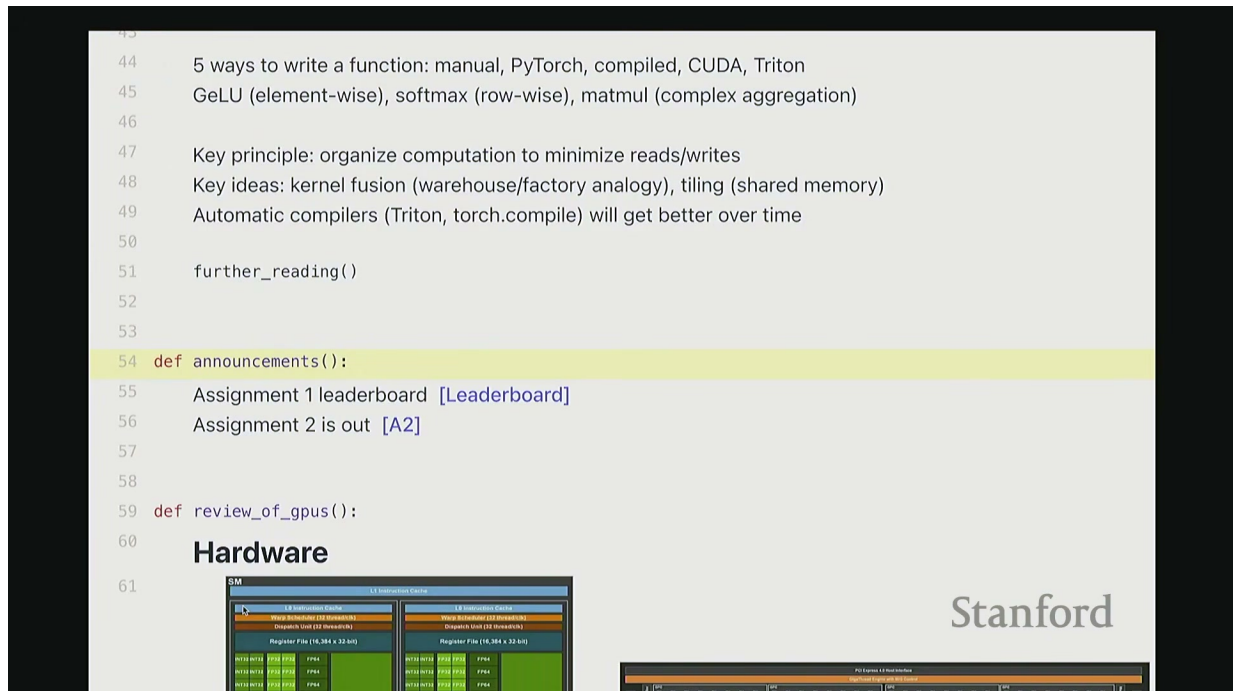


图 1: 课程列出的五种实现路线及共同目标。

视频证据时间窗: 00:01:25–00:02:05。

端到端时间可拆为

$$T_{\text{op}} = T_{\text{launch}} + T_{\text{memory}} + T_{\text{compute}} + T_{\text{sync}}.$$

- T_{launch} : 从主机提交 kernel 的固定开销;
- T_{memory} : 全局显存与片上存储之间的数据移动;
- T_{compute} : 算术指令和特殊函数单元时间;
- T_{sync} : 设备同步、流依赖及测量屏障。

先问是否值得手写

若算子已落入高度优化的 GEMM、卷积或厂商 fused kernel，手写版本很难胜出。最有价值的目标通常是形状特殊、融合机会明确、框架产生多次中间写回，或现有实现缺少所需精度与布局。

路线	开发成本	优化空间	适用情形
PyTorch 组合 自动编译	最低 低	依赖调度器 自动融合与生成	验证数学、建立基线 形状稳定的 Python 图
Triton	中	程序实例、分块、融合	自定义张量 kernel
CUDA 扩展	高	线程、内存和指令 全控制	极致性能或特殊硬件
厂商库	调用低、开发极高	架构专家优化	标准高价值算子

若候选实现时间为 T_i ，相对基线 T_0 的加速为

$$S_i = \frac{T_0}{T_i}.$$

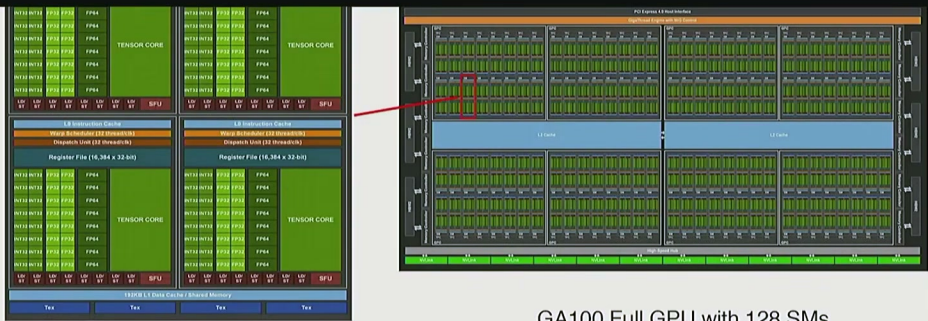
- T_0 : 同输入、同精度和同设备上的可信基线;
- T_i : 候选实现的稳态统计时间;
- 只有输出误差在阈值内时，加速比才有意义。

1.2 本章小结

kernel 工程不是从 CUDA 语法开始，而是从等价语义、代表性形状和基线开始。不同实现路线共享同一个评价函数：正确、快、可解释、可维护。

2 硬件执行与并行度

2.1 SM、线程块和尾波



The diagram illustrates the internal structure of a GPU. On the left, a detailed view of a Streaming Multiprocessor (SM) is shown, including Tensor Cores, FP64, FP32, and INT32 units, as well as L2 and L1 caches. On the right, a larger view shows the full GPU architecture with 128 SMs, L2 cache, and L1 cache. A red arrow points from the SM diagram to the full GPU diagram.

GA100 Full GPU with 128 SMs

SM

62 Compute: streaming multiprocessors (SMs) [A100: 108]
 63 Memory:
 64 • DRAM [A100: 80GB] - big, slow
 65 • L2 cache [A100: 40MB]
 66 • L1 cache [A100: 192KB per SM] - small, fast
 67
 68 You can look at the specs on your actual GPU.
 69 `print_gpu_specs()`
 70
 71 Basic structure: run $f(i)$ for all $i = 0, \dots, N-1$

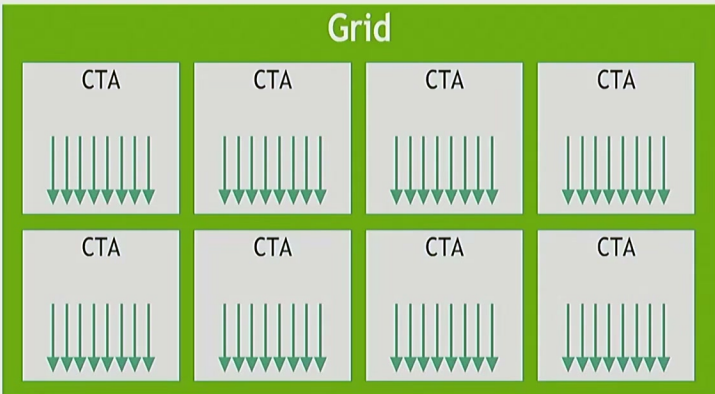
Stanford

图 2: GPU 由多个 SM、全局显存和片上存储组成。

视频证据时间窗: 00:02:25-00:03:05。

72
73
74

Execution model



The diagram shows a 2x4 grid of CTA (Concurrent Thread Arrays) blocks. Each block contains a series of downward-pointing arrows, representing threads. The entire grid is labeled 'Grid'.

75 • *Thread*: process individual index (i.e., $f(i)$)
 76 • *Thread block* (a.k.a. concurrent thread arrays): scheduled on a single SM
 77 • *Grid*: collection of thread blocks
 78
 79 Why thread blocks? Shared memory.

Stanford

图 3: grid 由多个 CTA/thread block 构成，每块调度到一个 SM。

视频证据时间窗: 00:03:05-00:03:45。

若有 G 个线程块、 M 个 SM，并假设一波每个 SM 执行一个块，则

$$N_{\text{wave}} = \left\lceil \frac{G}{M} \right\rceil.$$

- G : 问题规模和 block tile 共同决定的块数;
- M : 设备 SM 数量;
- 最后一波不足 M 块时形成尾波空闲。

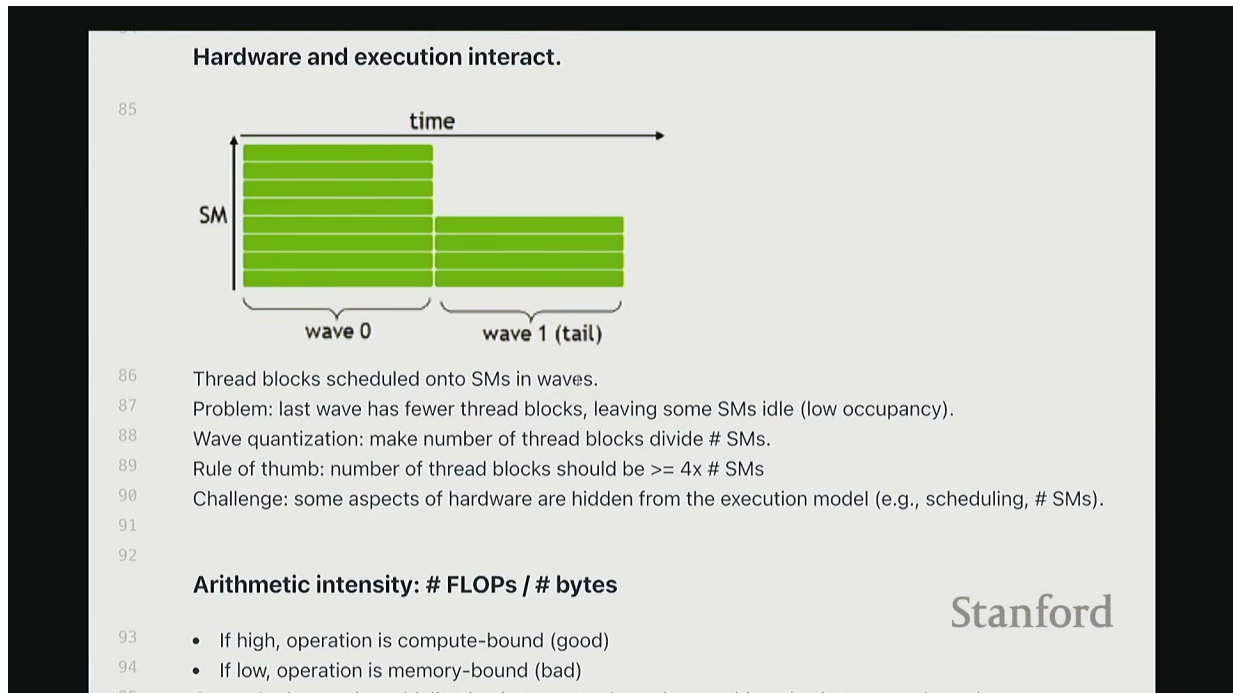


图 4: 线程块按波调度，最后一波可能只占用部分 SM。

视频证据时间窗: 00:04:10–00:04:50。

课程给出的经验规则是让线程块数明显多于 SM 数，例如

$$G \gtrsim 4M.$$

- $4M$: 不是硬件定理，而是为调度和延迟隐藏留余量的经验值;
- block 太少时，即使单块很高效，整卡仍会空闲;
- block 太多通常可排队，但单块资源仍限制驻留数量。

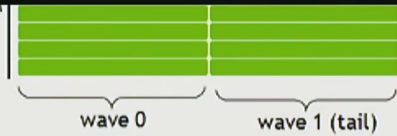
并行度有两层

第一层是网格是否给出足够多的 block; 第二层是每个 SM 能否同时驻留足够多的 warp。前者受问题切分影响，后者受寄存器、shared memory 和 block 线程数影响。两者缺一都会表现为低利用率。

算术强度仍是硬件判断的起点:

$$I = \frac{\text{FLOPs}}{\text{bytes}}.$$

- 高强度算子更可能受计算峰值约束；
- 低强度算子更可能受 HBM 带宽约束；
- 矩阵乘通常可通过分块得到高强度，逐元素算子通常不行。



86 Thread blocks scheduled onto SMs in waves.
 87 Problem: last wave has fewer thread blocks, leaving some SMs idle (low occupancy).
 88 Wave quantization: make number of thread blocks divide # SMs.
 89 Rule of thumb: number of thread blocks should be $\geq 4 \times \# \text{ SMs}$
 90 Challenge: some aspects of hardware are hidden from the execution model (e.g., scheduling, # SMs).
 91
 92

Arithmetic intensity: # FLOPs / # bytes

- If high, operation is compute-bound (good)
- If low, operation is memory-bound (bad)

95 General rule: matrix multiplication is compute-bound, everything else is memory-bound

96
 97

98 `def benchmarking_and_profiling():`
 99 IMPORTANT: benchmark/profile your code!
 100
 101 You can read spec sheets (marketing material) and papers

Stanford

图 5: 执行模型之后，课程立即回到算术强度和基准测试。

视频证据时间窗：00:05:25–00:06:05。

2.2 本章小结

一个 kernel 要快，既需要足够大的网格，也需要合理的单块资源。尾波、低驻留和低算术强度是三个不同问题，必须通过计数和 profiler 区分。

3 可信基准测试

3.1 异步执行为什么会骗过计时器

GPU kernel 启动通常是异步的。CPU 计时器包围函数调用，可能只测到排队时间。正确测量需要预热、重复执行，并在计时边界同步设备。


```

96
97
98 def benchmarking_and_profiling():
99     IMPORTANT: benchmark/profile your code!
100
101     You can read spec sheets (marketing material) and papers
102     ...but performance depends on your library version, your hardware, your workload
103     ...so there is no substitute for benchmarking/profiling your code.
104
105     Example computation: running forward/backward passes on an MLP.
106     run_mlp(dim=128, num_layers=16, batch_size=128, num_steps=5)
107
108     benchmarking()      # How long does it take?
109     profiling()         # Where time is being spent?
110
111     Every time you make a change, benchmark/profile!
112
113
114 class MLP(nn.Module):
115     """Simple MLP: linear -> GeLU -> linear -> GeLU -> ... -> linear -> GeLU"""
116     def __init__(self, dim: int, num_layers: int):
117         super().__init__()

```

Stanford

图 6: 课程编写统一 benchmark 函数, 比较端到端稳态时间。

视频证据时间窗: 00:07:10–00:07:50。

重复 n 次的均值为

$$\bar{t} = \frac{1}{n} \sum_{i=1}^n t_i.$$

- t_i : 第 i 次同步后的墙钟时间;
- n : 重复次数;
- 均值容易受抖动影响, 因此还应看中位数和分位数。

样本标准差为

$$s_t = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (t_i - \bar{t})^2}.$$

- s_t : 运行波动的尺度;
- 大波动可能来自温度、频率、后台任务、首次编译或缓存;
- 报告误差范围比只给三位小数更诚实。

```

233
234 def benchmark(description: str, run: Callable, num_warmups: int = 1, num_trials: int = 3):
235     """Benchmark `func` by running it `num_trials`, and return all the times."""
236     # Warmup: first times might be slower due to compilation, things not cached.
237     # Since we will run the kernel multiple times, the timing that matters is steady state.
238     for _ in range(num_warmups):
239         run()
240     if torch.cuda.is_available():
241         torch.cuda.synchronize() # Wait for CUDA threads to finish (important!)
242
243     # Time it for real now!
244     times: list[float] = [] # @inspect times, @inspect description
245     for trial in range(num_trials): # Do it multiple times to capture variance
246         start_time = time.time()
247
248         run() # Actually perform computation
249         if torch.cuda.is_available():
250             torch.cuda.synchronize() # Wait for CUDA threads to finish (important!)
251
252         end_time = time.time()
253         times.append((end_time - start_time) * 1000) # @inspect times
254

```

Stanford

图 7: 预热、同步、重复和统计构成可信计时协议。

视频证据时间窗: 00:11:25–00:12:05。

第一次运行不能代表稳态

JIT 编译、CUDA context 初始化、动态库加载和内存池创建都会污染首次测量。课程反复强调：每次改动都要重新 benchmark 和 profile，但不要把编译时间混进稳态 kernel 时间。

矩阵乘扫描的理论 FLOPs 为

$$F = 2mnk.$$

- m, n, k : 矩阵乘的三个维度;
- 用 F/t 可把不同形状统一成吞吐;
- 仍需报告形状, 因为相同 FLOPs 可能有完全不同的 tile 和尾波。

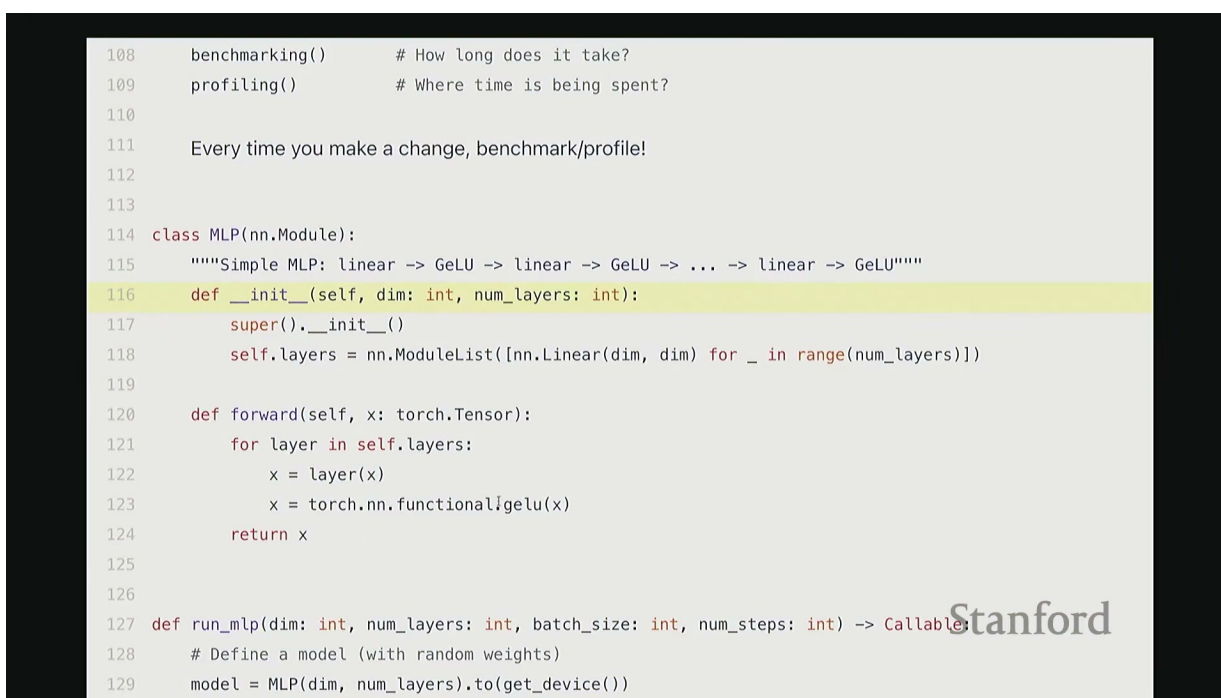


图 8: 方阵矩阵乘在不同尺寸上的基准结果。

视频证据时间窗: 00:09:10-00:09:50。

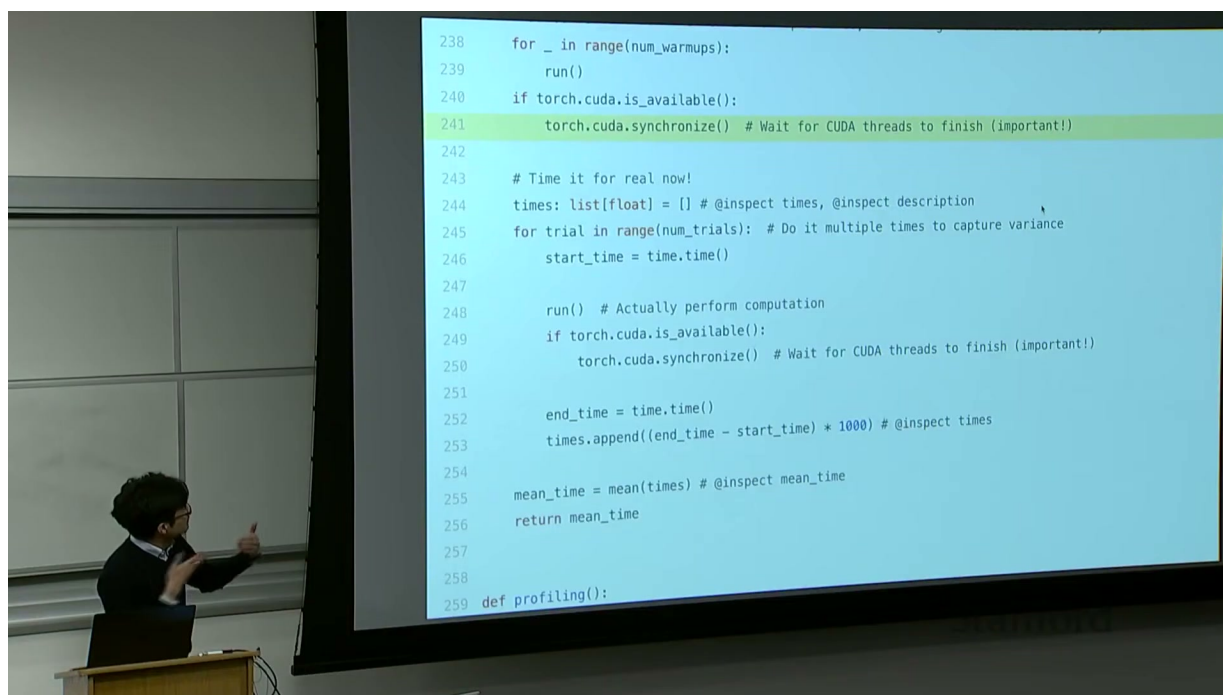


图 9: 课程扫描 1024、2048、4096、8192、16384 等规模。

视频证据时间窗: 00:13:05-00:13:45。

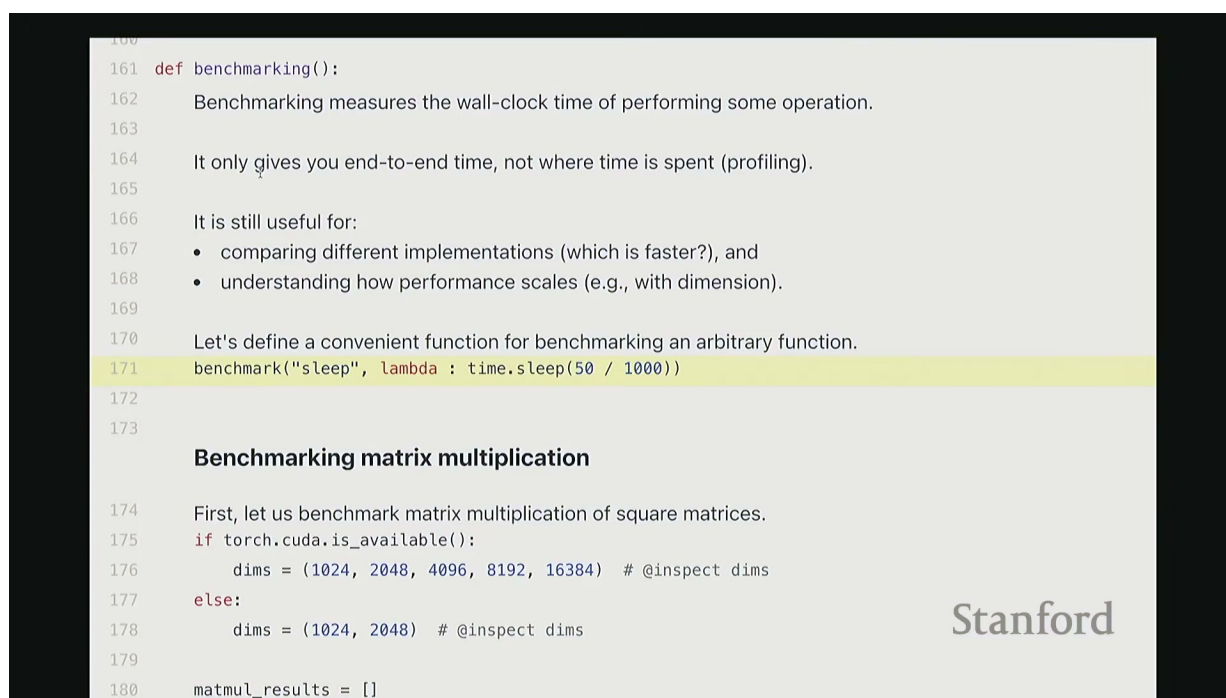


图 10: 将基准测试扩展到真实 MLP 形状与批量。

视频证据时间窗: 00:14:05–00:14:45。

有效吞吐为

$$P_{\text{eff}} = \frac{F}{t}.$$

- F : 约定口径下的操作数;
- t : 稳态执行时间;
- 比较不同实现时必须使用同一 FLOPs 口径。

基准测试矩阵要覆盖工作负载

至少覆盖短小 kernel、典型训练形状、边界不整除形状和极大形状。只测一个漂亮二次幂，常会高估实际收益；只测端到端，又可能看不出被其他算子淹没的局部变化。

3.2 本章小结

可信基准的核心是控制变量：同输入、同精度、正确同步、充分预热、重复统计和形状覆盖。数字若不可复现，就不能指导优化。

4 从表格到时间线: Profiler 推理

4.1 逐元素算子为什么慢

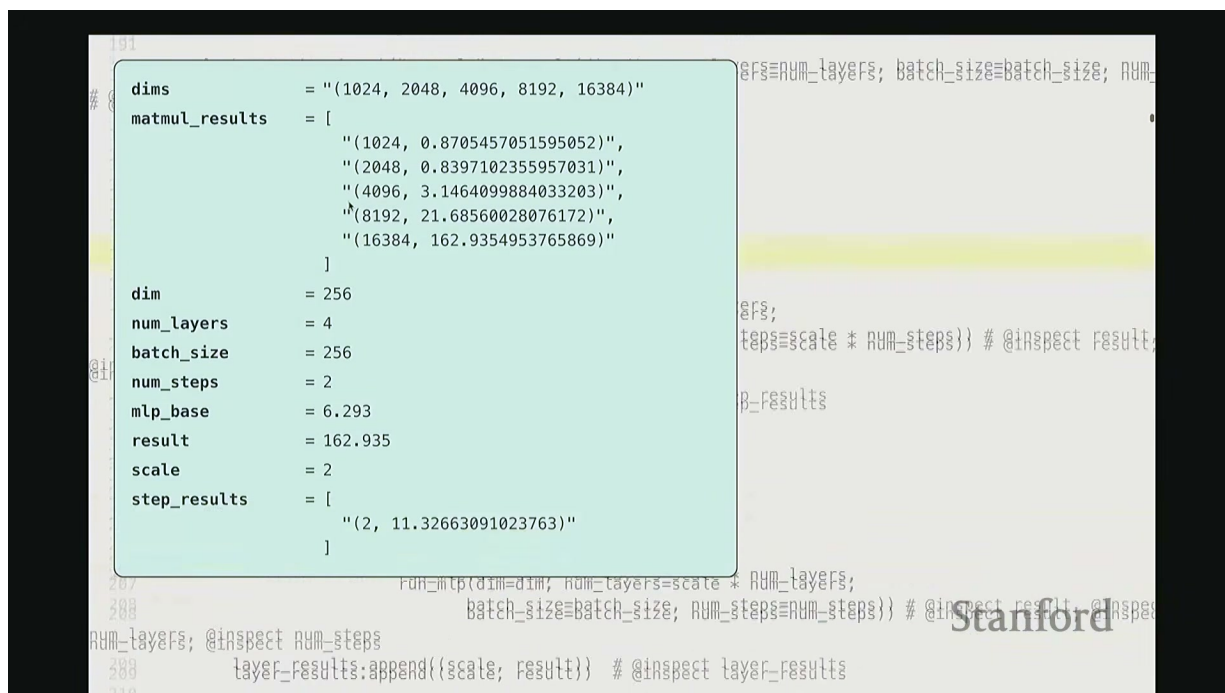


图 11: add、matmul 等算子展示不同的计算/带宽行为。

视频证据时间窗: 00:15:40-00:16:20。

对一次读两个输入、写一个输出的 FP32 加法, 算术强度近似

$$I_{\text{add}} = \frac{1}{4 + 4 + 4} = \frac{1}{12}.$$

- 分子是一条浮点加法;
- 分母是两个 4 字节输入和一个 4 字节输出;
- 如此低的强度意味着性能主要取决于内存流量。

```

249     if torch.cuda.is_available():
250         torch.cuda.synchronize() # Wait for CUDA threads to finish (important!)
251
252     end_time = time.time()
253     times.append((end_time - start_time) * 1000) # @inspect times
254
255     mean_time = mean(times) # @inspect mean_time
256     return mean_time
257
258
259 def profiling():
260     While benchmarking looks at end-to-end time, profiling looks at where time is spent.
261     Obvious: profiling helps you understand where time is being spent.
262     Deeper: profiling helps you understand (what is being called).
263
264     PyTorch has a nice built-in profiler https://pytorch.org/tutorials/recipes/recipes/profiler\_recipe.html
265
266     Let's profile some code to see what is going on under the hood.
267     sleep_function = lambda : time.sleep(50 / 1000)
268     sleep_profile = profile("sleep", sleep_function)
269
270     sleep

```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	# of Calls
cudaDeviceSynchronize	100.00%	11.610us	100.00%	11.610us	5.805us	2

Stanford

图 12: PyTorch Profiler 将 CPU 提交与 GPU kernel 映射到时间线。

视频证据时间窗: 00:18:25-00:19:05。

```

Name Self CPU % Self CPU CPU total % CPU total CPU time avg # of Calls

cudaDeviceSynchronize 100.00% 11.610us 100.00% 11.610us 5.805us 2

Self CPU time total: 11.610us

Let's start with some basic operations.
add_function = lambda a, b: a + b
add_profile = profile("add", run_operation2(dim=2048, operation=add_function))

add

```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self CUDA	Self CUDA %	CUDA total	CUDA time avg	# of Calls
aten::add	98.02%	1.392ms	99.38%	1.411ms	1.411ms	17.119us	100.00%	17.119us	17.119us	1
void aten::native::vectorized_elementwise_kernel<4, aten::CUDAFunctionAdd...	0.00%	0.000us	0.00%	0.000us	0.000us	17.119us	100.00%	17.119us	17.119us	1
cudaLaunchKernel	1.37%	19.392us	1.37%	19.392us	19.392us	0.000us	0.00%	0.000us	0.000us	1
cudaDeviceSynchronize	0.62%	8.734us	0.62%	8.734us	4.367us	0.000us	0.00%	0.000us	0.000us	2

Self CPU time total: 1.428ms Self CUDA time total: 17.119us

```

matmul_function = lambda a, b: a @ b
matmul_profile = profile("matmul", run_operation2(dim=2048, operation=matmul_function))

matmul

```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self CUDA	Self CUDA %	CUDA total	CUDA time avg	# of Calls
aten::matmul	2.29%	7.520us	97.24%	318.634us	318.634us	0.000us	0.00%	342.620us	342.620us	1
aten::mm	98.14%	295.387us	94.94%	311.114us	311.114us	342.620us	100.00%	342.620us	342.620us	1
void cutlass::Kernel2<cutlass::B0...	0.00%	0.000us	0.00%	342.620us	342.620us	342.620us	100.00%	342.620us	342.620us	1
cudaDeviceGetAttribute	0.21%	0.690us	0.21%	0.690us	0.690us	0.000us	0.00%	0.000us	0.000us	1

Stanford

图 13: 展开 GPU 轨道后可检查真实 kernel 次序、间隙和重叠。

视频证据时间窗: 00:21:25-00:22:05。

Profiler 的三步阅读法

先看端到端哪一段最长；再展开该段对应的 GPU kernel，区分计算、memcpy 和空闲；最后回到调用栈定位产生它的 PyTorch 算子。不要从数百条 kernel 名字中凭感觉寻找“可疑项”。

若一串 k 个逐元素算子不融合，理想 HBM 流量约为

$$Q_{\text{sep}} \approx 2ksN.$$

- s : 每个元素字节数;
- N : 元素数量;
- 每个阶段写回再读入，流量与阶段数线性增长。

融合后可接近

$$Q_{\text{fused}} \approx (r + w)sN.$$

- r : 从 HBM 读取的独立输入张量数;
- w : 最终写出的结果张量数;
- 中间值停留在寄存器，不再形成完整张量。

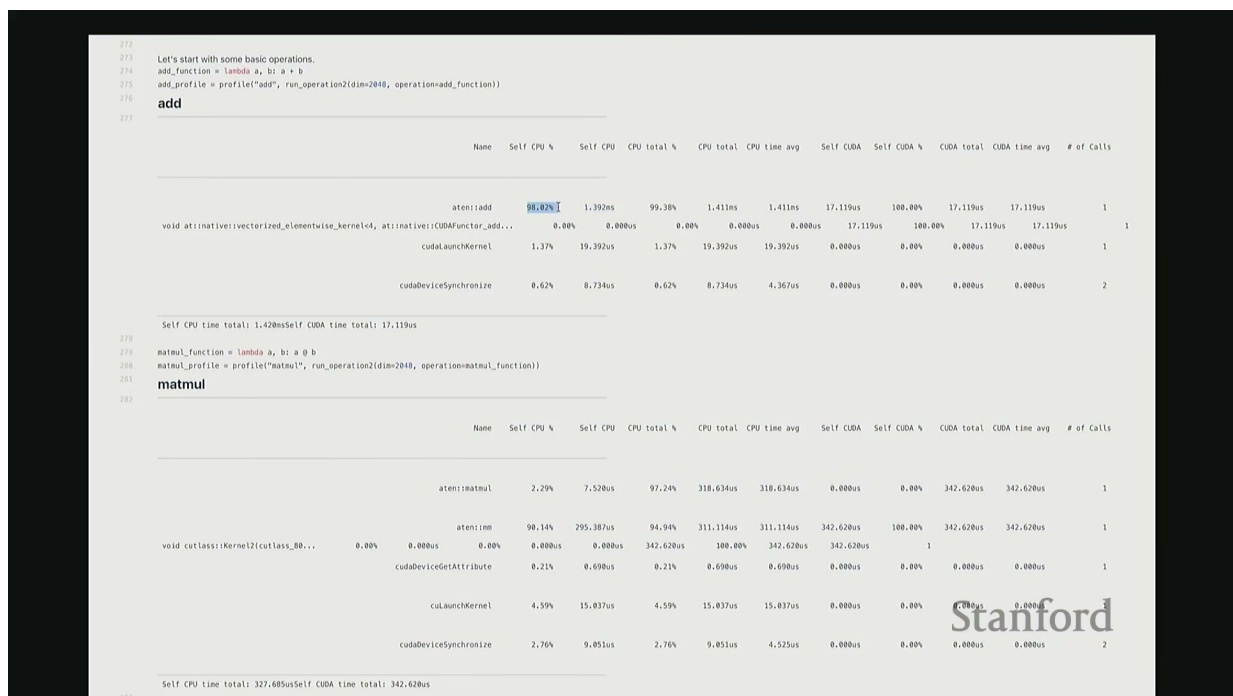


图 14: 融合减少 kernel 数，也减少中间张量读写。

视频证据时间窗：00:25:10-00:25:50。

时间线上的空白也是数据

GPU 轨道空白可能表示 CPU 没及时提交、同步过多、数据加载不足或图被切断。只有 kernel 内部慢时才值得继续看指令、访存和占用；否则应先修主机和图级调度。

4.2 本章小结

benchmark 告诉我们“慢多少”，时间线告诉我们“慢在哪里”。将算术强度、调用栈和 GPU 轨道结合，才能决定下一步是融合、重排、并发还是手写 kernel。

5 Nsight 与底层瓶颈

5.1 从 kernel 时间进入硬件计数器

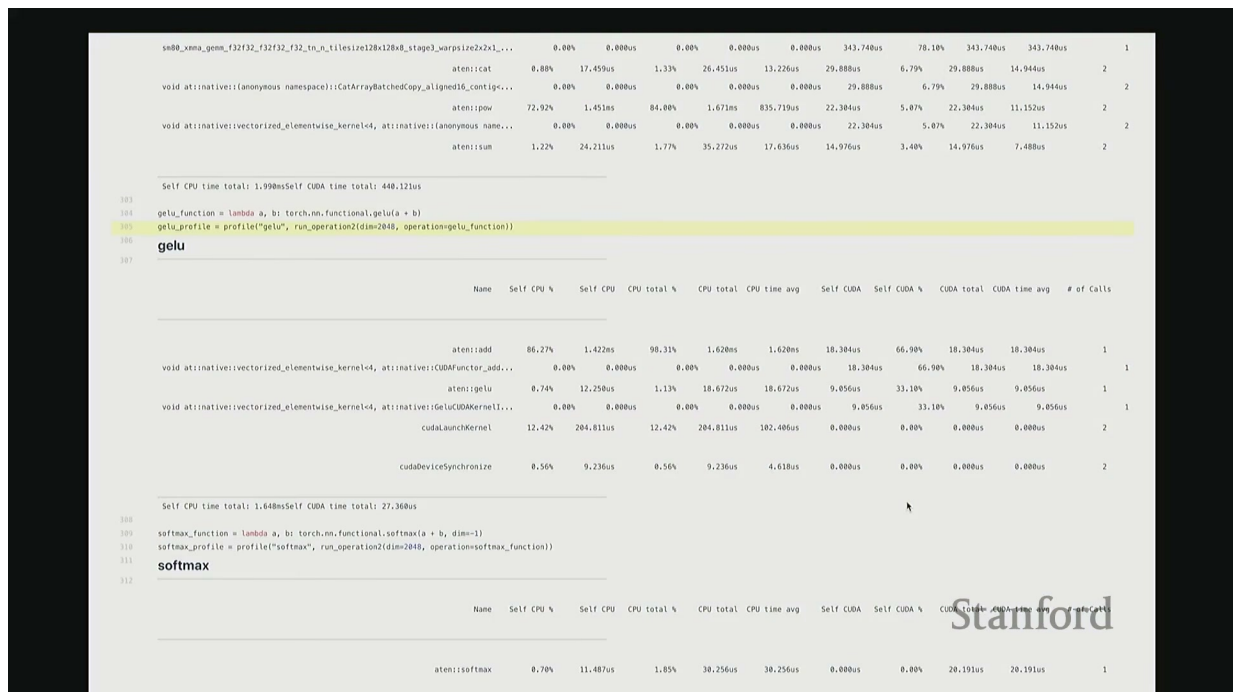


图 15: Nsight 展示 kernel、内存事务和硬件利用率。

视频证据时间窗：*00:30:10-00:30:50*。

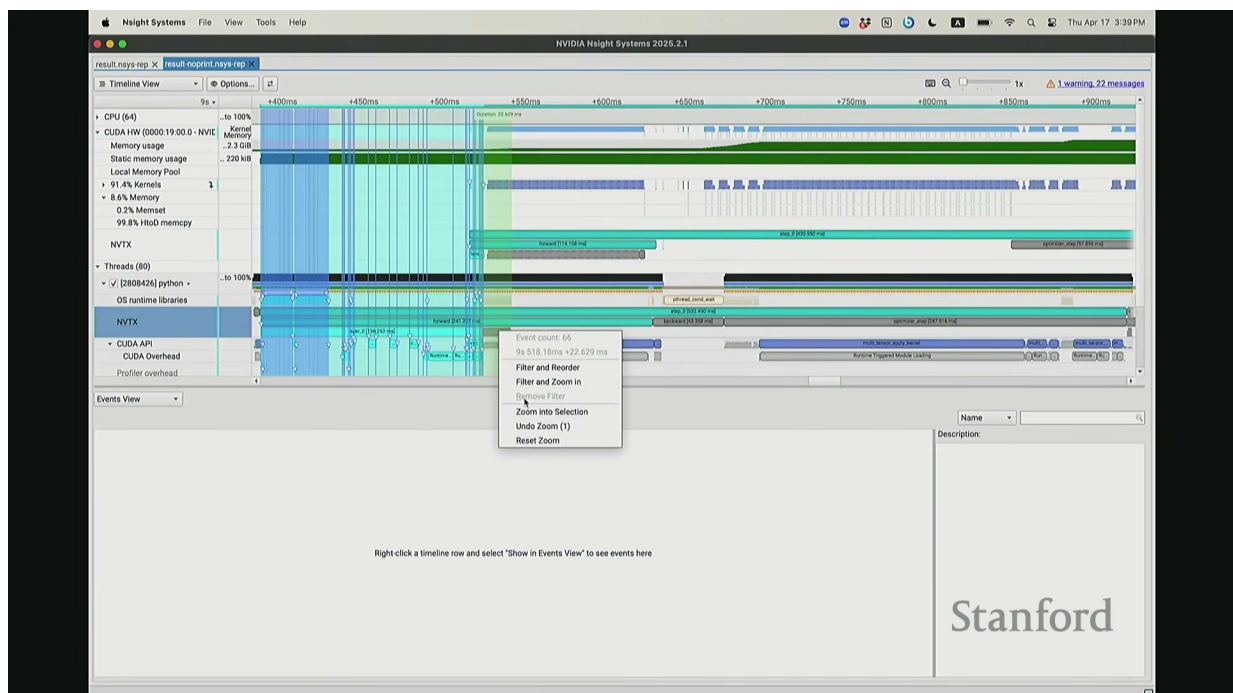


图 16: 放大时间线后检查 kernel 并发、同步和空泡。

视频证据时间窗：00:37:05-00:37:45。

若测得 DRAM 流量为 Q 、时间为 t ，有效带宽是

$$B_{\text{eff}} = \frac{Q}{t}.$$

- Q : 由 profiler 计数器给出的实际读写字节;
- t : kernel 持续时间;
- 与设备可持续带宽比较, 可判断是否已接近带宽屋顶。

若完成 F FLOPs, 则计算利用率可写为

$$u_{\text{compute}} = \frac{F/t}{P_{\text{peak}}}.$$

- P_{peak} : 相同数据类型和指令路径下的峰值;
- F/t : 实测有效吞吐;
- 低计算利用率不一定是算术问题, 也可能是带宽或并行度不足。

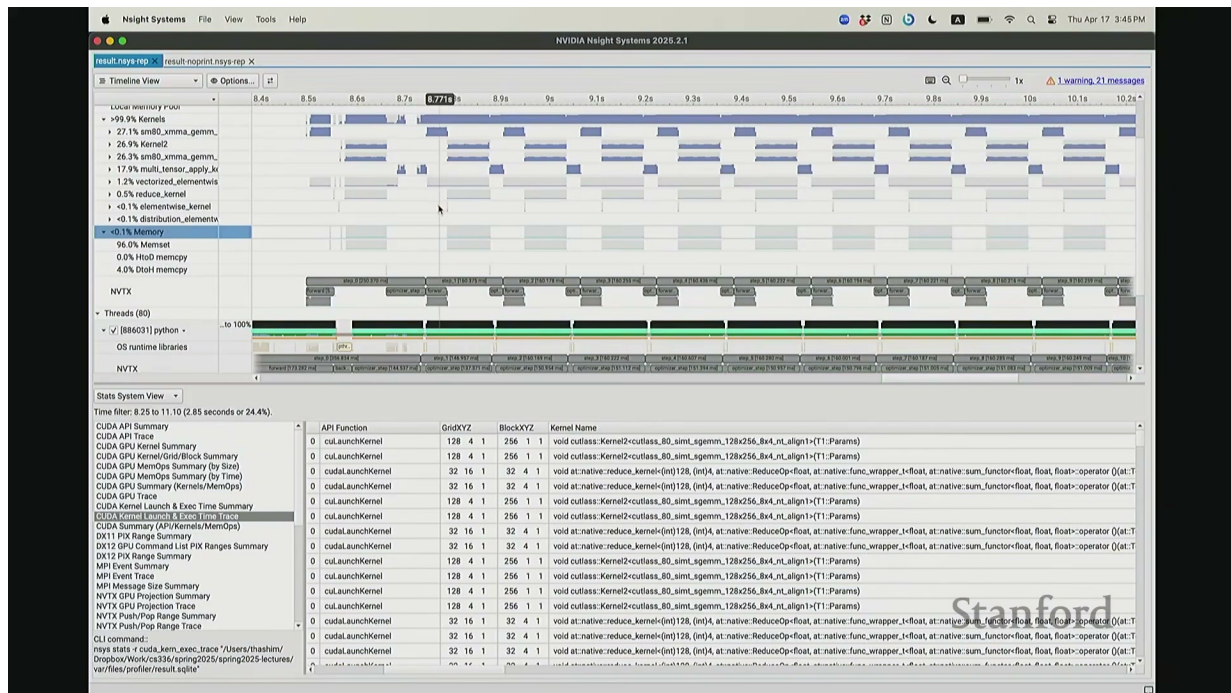


图 17: 用 memory-bound 与 compute-bound 模型组织硬件计数器。

视频证据时间窗: 00:43:05-00:43:45。

计数器需要上下文

缓存命中率高可能是好事, 也可能说明工作集太小; 高 occupancy 不代表指令持续发射; Tensor Core 利用率低可能源于形状、布局或精度。任何单一百分比都不能独立证明根因。

理论最短时间由两种屋顶共同约束:

$$T_{\min} \geq \max\left(\frac{F}{P_{\text{peak}}}, \frac{Q}{B_{\text{peak}}}\right).$$

- 第一项是计算下界;

- 第二项是带宽下界；
- 实测远高于两者时，继续检查调度、依赖、指令混合与尾波。

优化建议必须能被反证

例如“该 kernel 带宽受限”应预测：降低字节数会变快，提高纯计算峰值不会显著变快。做一次融合或精度对照即可检验。把 profiler 读数转换为可测试预测，比罗列计数器更有用。

5.2 本章小结

Nsight 将高级算子继续分解成带宽、计算、占用和调度证据。真正的目标不是把每个指标变绿，而是找到限制 $T_{\min}$ 的主导项并做可反证实验。

6 CUDA 扩展：控制权与工程成本

6.1 Python、C++ glue 与 <<<grid, block>>>

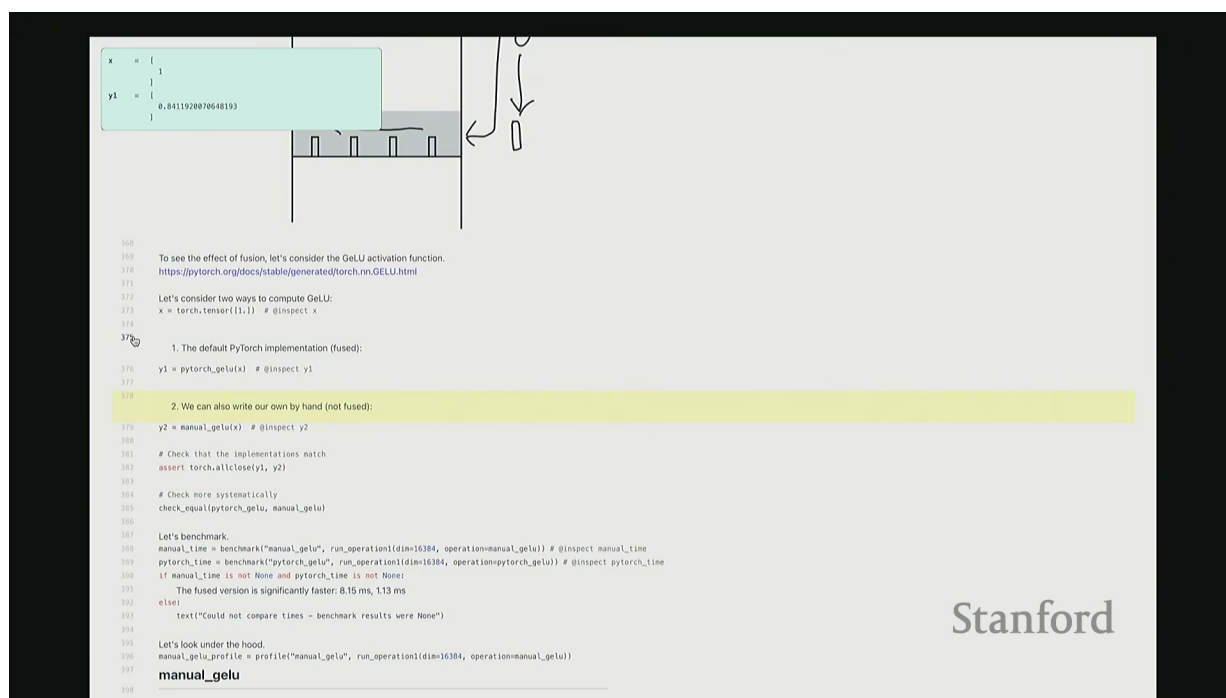


图 18: 先用 PyTorch 组合建立正确性与性能基线。

视频证据时间窗：00:45:30–00:46:10。

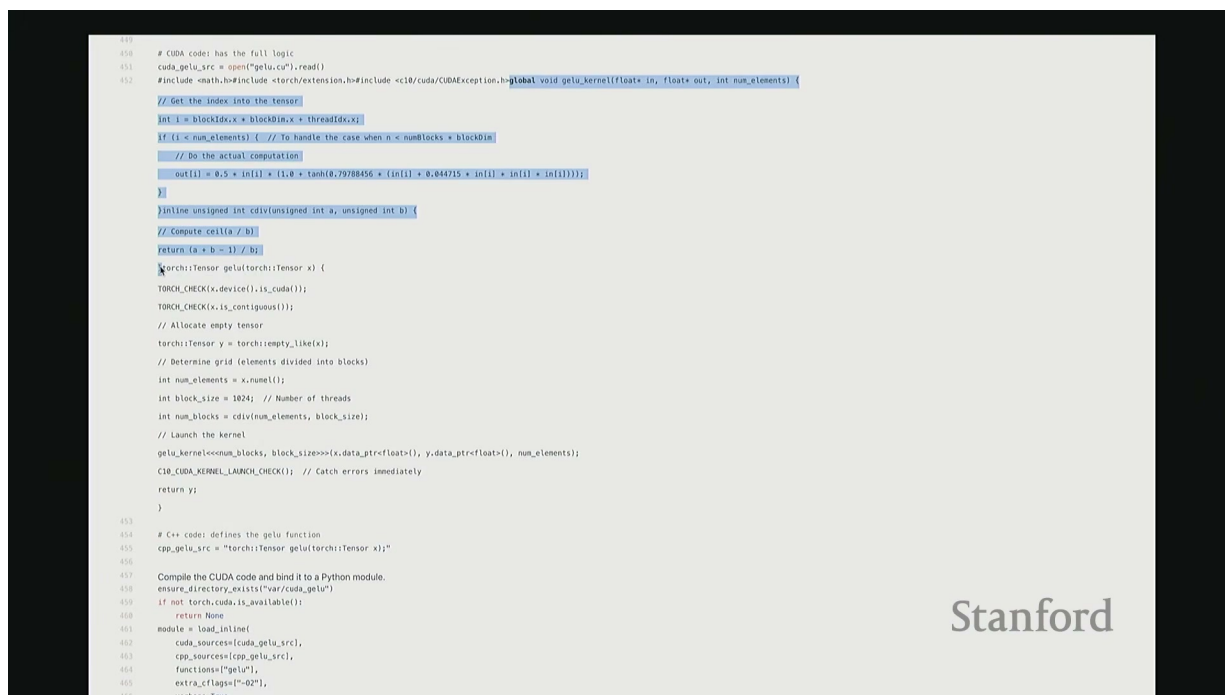


图 19: CUDA 扩展需要张量检查、分配、索引和构建 glue。

视频证据时间窗: 00:50:40–00:51:20。

一维逐元素 kernel 常用全局索引

$$i = \text{blockIdx.x} \cdot B + \text{threadIdx.x}.$$

- B : 每个 block 的线程数;
- blockIdx.x : 块编号;
- threadIdx.x : 块内线程编号;
- 当 $i \geq N$ 时必须通过边界判断避免越界。

所需 block 数为

$$G = \left\lceil \frac{N}{B} \right\rceil.$$

- N : 总元素数;
- G : 启动网格大小;
- 向上取整产生的多余线程由边界条件屏蔽。

```

cuda_gelu_src = open("gelu.cu").read()
#include <math.h>#include <torch/extension.h>#include <c10/cuda/CUDAException.h>global void gelu_ke

// Get the index into the tensor
int i = blockIdx.x * blockDim.x + threadIdx.x;

if (i < num_elements) { // To handle the case when n < numBlocks * blockDim

    // Do the actual computation

    out[i] = 0.5 * in[i] * (1.0 + tanh(0.79788456 * (in[i] + 0.044715 * in[i] * in[i] * in[i]))));

}

inline unsigned int cdiv(unsigned int a, unsigned int b) {

// Compute ceil(a / b)

return (a + b - 1) / b;

}torch::Tensor gelu(torch::Tensor x) {

TORCH_CHECK(x.device().is_cuda());

TORCH_CHECK(x.is_contiguous());

// Allocate empty tensor

torch::Tensor y = torch::empty_like(x);

```

Stanford

图 20: CUDA 启动配置把问题规模映射为 grid 和 block。

视频证据时间窗: 00:54:25–00:55:05。

Listing 1: CUDA 逐元素 kernel 的结构化伪代码

```

1 __global__ void gelu_kernel(const float* x, float* y, int n) {
2     int i = blockIdx.x * blockDim.x + threadIdx.x;
3     if (i < n) {
4         float v = x[i];
5         y[i] = 0.5f * v * (1.0f + tanhf(...));
6     }
7 }

```

手写 CUDA 的正确性清单

检查 dtype、device、contiguous/stride、元素数量、边界、当前 CUDA stream、错误传播和数值容差。课程画面中的 glue code 很长，恰好说明“真正计算的几行”只占扩展工程的一小部分。

若最大绝对误差为

$$e_{\infty} = \max_i |y_i - \hat{y}_i|$$

- y_i : 参考实现结果;
- $\hat{y}_i$: 候选 kernel 结果;
- 容差应按精度和数值范围设定，不能用“看起来接近”代替测试。

6.2 本章小结

CUDA 给出最完整的线程和内存控制，也要求开发者承担张量接口、构建、流语义和边界安全。只有当 profiler 证据指向底层控制不足时，这个成本才合理。

7 Triton: 块级 SPMD 编程

7.1 程序实例而不是单线程

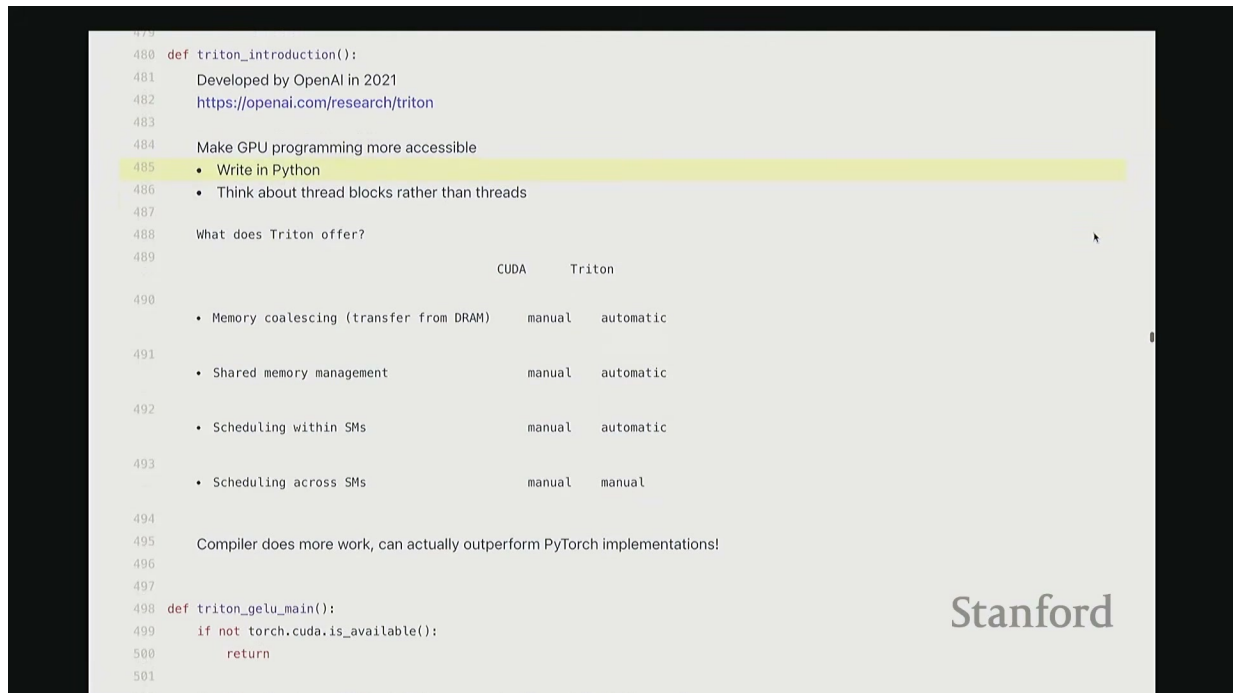


图 21: Triton 在 Python 中表达块级程序，由编译器完成线程级细节。

视频证据时间窗：01:02:10–01:02:50。

Triton 用 `program_id` 标识程序实例。若每个实例处理 B 个元素，则偏移向量为

$$\mathbf{o} = pB + [0, 1, \dots, B - 1].$$

- p : 当前 program id;
- B : 编译期块大小;
- $\mathbf{o}$: 该实例负责的一组全局元素索引。

边界掩码为

$$\mathbf{m} = (\mathbf{o} < N).$$

- N : 有效元素总数;
- 掩码控制 `tl.load` 和 `tl.store`;
- 它让块大小可取方便编译的二次幂而不越界。

```

550
551 @triton.jit
552 def triton_gelu_kernel(x_ptr, y_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
553     # Input is at 'x_ptr' and output is at 'y_ptr'
554     # | x_ptr | Block 0 | Block 1 | ... |
555     # |-----| BLOCK_SIZE | num_elements
556
557     pid = tl.program_id(axis=0)
558     block_start = pid * BLOCK_SIZE
559
560     # Indices where this thread block should operate
561     offsets = block_start + tl.arange(0, BLOCK_SIZE)
562
563     # Handle boundary
564     mask = offsets < num_elements
565
566     # Read
567     x = tl.load(x_ptr + offsets, mask=mask)
568
569     # Approx gelu is 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715 * x^3)))
570     # Compute (tl.tanh doesn't exist, use tanh(a) = (exp(2a) - 1) / (exp(2a) + 1))
571     a = 0.79788456 * (x + 0.044715 * x * x * x)
572     exp = tl.exp(2 * a)
573     tanh = (exp - 1) / (exp + 1)
574     y = 0.5 * x * (1 + tanh)
575
576     # Store
577     tl.store(y_ptr + offsets, y, mask=mask)
578

```

Stanford

图 22: 多个 Triton program instance 分块覆盖输入。

视频证据时间窗: 01:05:00-01:05:40。

Listing 2: Triton 逐元素 GELU 的核心结构

```

1 @triton.jit
2 def gelu_kernel(x_ptr, y_ptr, n: tl.constexpr, BLOCK: tl.constexpr):
3     pid = tl.program_id(0)
4     offsets = pid * BLOCK + tl.arange(0, BLOCK)
5     mask = offsets < n
6     x = tl.load(x_ptr + offsets, mask=mask)
7     y = 0.5 * x * (1.0 + ...)
8     tl.store(y_ptr + offsets, y, mask=mask)

```

Triton 的抽象边界

开发者仍需决定分块、访问顺序、warp 数和布局，但不必手工写线程级地址计算与 C++ glue。编译器拥有块内全局视野，可以做向量化、指令调度和访存合并；这正是它比“在 Python 中模拟 CUDA”更高层的地方。

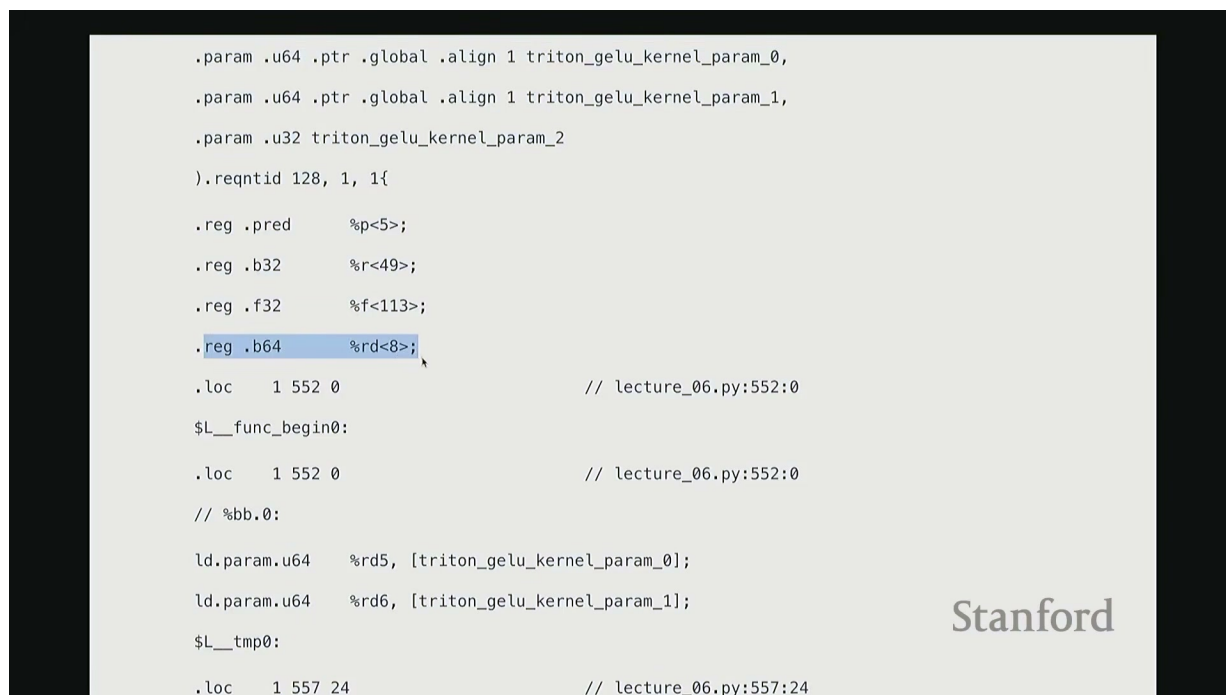


图 23: 检查 Triton 生成的 PTX, 可验证编译器采用了哪些低层指令。

视频证据时间窗: 01:08:25–01:09:05。

若 Triton 版本时间为 t_{triton} 、CUDA 版本为 t_{cuda} , 相对差距为

$$\delta = \frac{t_{\text{triton}} - t_{\text{cuda}}}{t_{\text{cuda}}}.$$

- $\delta < 0$: Triton 更快;
- $\delta > 0$: CUDA 更快;
- 结论只能在相同形状、精度和预热条件下解释。

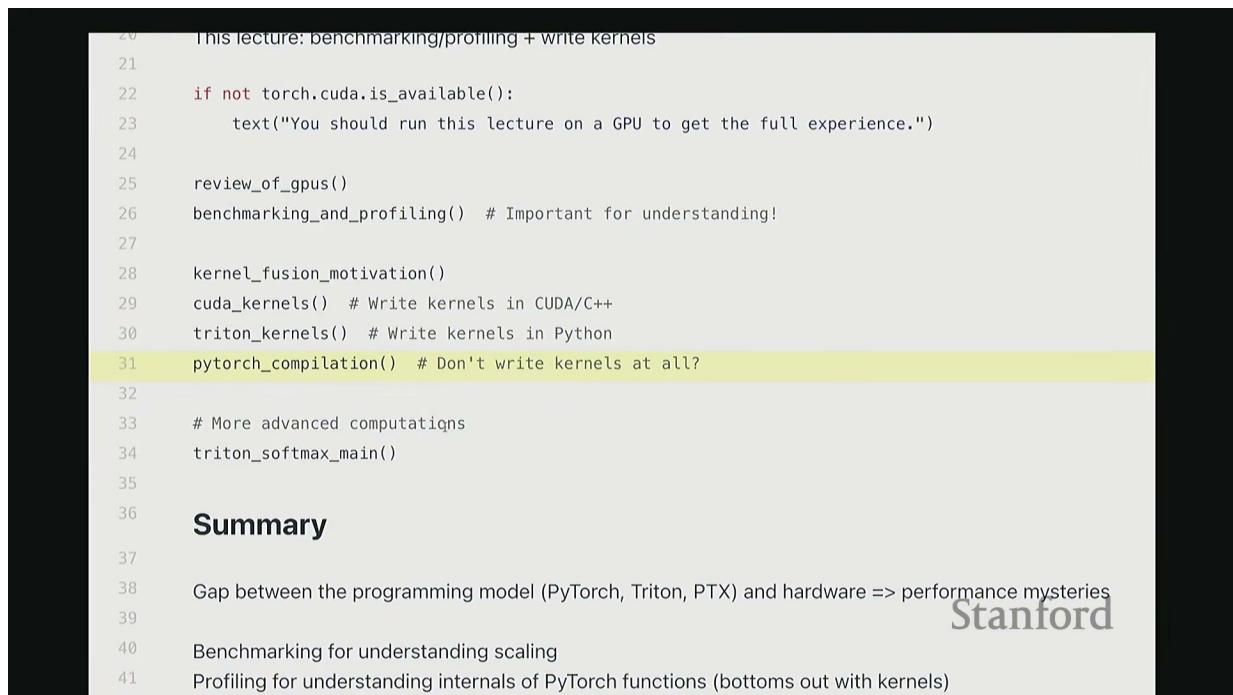


图 24: 课程比较手写、PyTorch、CUDA、Triton 与编译版本。

视频证据时间窗: 01:13:25–01:14:05。

需要读生成代码的时刻

当性能结果无法用 Roofline、形状或流量解释时，查看 PTX/SASS：是否使用向量加载、是否产生 local-memory spill、是否进入预期特殊函数或 Tensor Core 路径。高层源码相同，不代表编译结果相同。

7.2 本章小结

Triton 把优化单位从单线程提升到张量块。它降低了写自定义 kernel 的门槛，却没有替我们选择正确 tile、mask、布局 and 性能实验。

8 Softmax：归约 kernel 的完整案例

8.1 行级分块与稳定归一化

对一行 $x \in \mathbb{R}^n$ ，稳定 softmax 为

$$m = \max_i x_i, \quad z_i = e^{x_i - m}, \quad y_i = \frac{z_i}{\sum_j z_j}.$$

- m : 行最大值，避免指数溢出；
- z_i : 平移后的指数；
- 分母：同一行的归约和；
- 一次 program 可负责一整行，在片上完成两次归约。

```

640
641 def triton_softmax_main():
642     So far, we've looked at elementwise operations in Triton (e.g., GeLU).
643     Now let us look at operations that aggregate over multiple values.
644
645     We will roughly follow the Triton fused softmax tutorial: https://triton-lang.org/main/getting-started/tutorials/02-fused-softmax.html
646
647     Recall the softmax operation is used in attention and generating probabilities.
648     Normalize each row of a matrix:
649     [A1 A2 A3] => [A1/A A2/A A3/A]
650     [B1 B2 B3] => [B1/B B2/B B3/B]
651
652     Let's first start with the naive implementation and keep track of reads/writes.
653     x = torch.tensor([
654         [5., 5, 5],
655         [0, 0, 100],
656     ], device=get_device())
657     y1 = manual_softmax(x) # @inspect y1
658
659     if not torch.cuda.is_available():
660         return

```

Stanford

图 25: Triton softmax 让每个 program 处理一行，并用 mask 填充到块大小。

视频证据时间窗：01:15:25–01:16:05。

若块大小选择为不小于 n 的最小二次幂，

$$B = 2^{\lceil \log_2 n \rceil}.$$

- n : 真实列数;
- B : 编译期 block size;
- $B - n$ 个位置通过 mask 设为 $-\infty$ ，不影响最大值和指数和。

朴素三算子实现至少形成多次中间写回，融合后每行理想主存流量接近

$$Q_{\text{row}} \approx 2ns.$$

- s : 元素字节数;
- 一次读取 logits、一次写出概率;
- 最大值和指数和保留在片上，不写成全尺寸中间张量。

softmax 的优化顺序

先确保减最大值的数值稳定性，再选择一行一程序还是多程序协作；然后根据 n 调块大小和 warp 数；最后扫描短行、长行与非整齐形状。正确性、布局 and 性能必须一起回归。

实现	优势	隐含工作	常见失败
PyTorch 组合 自动编译	最清晰的参考语义 低改动自动融合	多 kernel 和中间张量 图捕获、特化和编译	带宽与 launch 动态形状回退
Triton	块级归约表达简洁	tile、mask、warp 调 优	长行资源压力
CUDA	控制最完整	glue、构建、线程归约	高维护成本

不要把一个胜利形状推广到所有输入

softmax 的最佳策略随行长变化。短行可能受 launch 限制，中等行适合单 program，极长行需要多块归约。生产实现通常包含多条特化路径和回退，而不是一个固定 BLOCK。

本讲留下的工程闭环

写参考实现与随机测试；定义真实形状矩阵；预热、同步、重复 benchmark；用 profiler 判断图级或 kernel 级瓶颈；改变一个因素并验证预测；检查生成代码；把正确性和性能阈值加入回归。

8.2 本章小结

softmax 展示了 Triton 的核心价值：用块级 SPMD 表达融合归约，让中间状态停留片上。它也展示了边界：块大小、行长、warp、寄存器和数值精度仍需要工程判断。会写 kernel 的标志不是代码短，而是能从测量证据解释它为什么快、何时会慢。

9 Kernel 工程复盘：从作业到生产

9.1 建立语义契约

自定义 kernel 的输入契约至少包含 shape、stride、dtype、device、对齐和是否允许非连续张量；输出契约还要说明原地修改、别名关系、数值误差和梯度。课程示例为了集中解释性能，常假设连续 CUDA 张量和固定 dtype。生产代码必须显式检查这些假设，或提供可靠回退。隐式假设最危险，因为它在常用输入上表现正常，在长尾输入上才造成越界或静默错误。

正确性测试不应只生成标准正态随机数。还要覆盖零、正负极值、非常接近的 logits、NaN/Inf 策略、长度为零或一、刚超过 BLOCK 的边界、不能整除 warp 的形状和带奇怪 stride 的切片。若算子可求导，应分别做前向参考对照、解析梯度对照和有限差分抽查。不同精度应采用不同绝对与相对容差，并解释容差来自何种累加顺序。

参考实现也要固定口径

参考实现必须和候选 kernel 使用相同数学公式、广播规则和精度。若参考在 FP32 计算而候选在 BF16 计算，误差同时包含算法与精度差异；若 GELU 一个用 exact、一个用 tanh approximation，则比较本身无效。

9.2 设计 benchmark 套件

将测试形状分成三类：课堂代表形状用来复现实验，工作负载形状来自真实训练日志，边界形状专门攻击 tile 和 mask。每类都保存输入字节、理论 FLOPs、基线时间、候选时间和显存峰值。短算子要增加重复次数并使用 CUDA event 或同步计时；长算子要控制温度和功耗状态；JIT 路线则同时报告首次编译与稳态执行，但不能混成一个数字。

对结果做分层归因：首先看端到端训练能节省多少；其次看算子总时间；再次看具体 kernel。若某候选在单 kernel 上更快，却导致图断裂、额外 contiguous copy 或同步，端到端仍可能退化。基准脚本应包含一次完整调用链，而不是只测脱离上下文的内核函数。

建议保存的统计

中位数、p90、最小值、标准差、有效带宽、有效 TFLOP/s、DRAM bytes、kernel 数、编译缓存命中、峰值显存和数值误差。最小值接近硬件潜力，中位数代表常态，p90 暴露抖动；三者承担不同解释角色。

9.3 Profiler 决策树

若 GPU 时间线有大块空白，先检查 CPU 提交、数据加载、同步和 graph break；若时间线密集但由许多短 kernel 组成，优先融合或编译；若单个 kernel 主导，计算其实际带宽和吞吐，再选择内存或计算方向。带宽接近屋顶时，尝试减少字节、提高复用或改变精度；计算接近屋顶时，重点是减少 FLOPs、进入更合适的专用指令或并行更多独立工作。

若两种屋顶都远未达到，检查网格是否足够大、单块是否因寄存器或 shared memory 限制驻留、是否存在长依赖链、分支和尾波。Nsight 指标用于区分候选根因，但每个根因都应产生一个小型对照实验。例如减半寄存器使用若提高占用却不提速，说明占用率不是主导瓶颈。

避免 profiler 过度采样

全量硬件计数器会多次重放 kernel，时间不再等同真实运行。先用轻量时间线定位，再对少数关键 kernel 收集针对性计数器。报告中区分“真实一次运行的时间”和“计数器重放产生的诊断数据”。

9.4 CUDA 路线的维护边界

CUDA 扩展需要处理构建工具链、编译架构列表、ABI、当前 stream、错误码和设备 guard。一次成功编译不代表可分发：换 GPU 架构可能需要重新生成 cubin 或保留 PTX，换 PyTorch 版本可能改变 C++ 接口，Windows/Linux 编译器也可能有差异。若性能收益只在少数形状出现，应保留动态 dispatch，让其他形状回到框架实现。

kernel 启动参数最好由输入和设备属性计算，而不是把课堂数字硬编码。BLOCK 大小影响边界浪费、warp 数、寄存器和共享内存；grid 大小影响并行度和尾波。每个特化配置都应有适用 shape 范围、性能证据和回退条件，否则配置数量会迅速变成不可维护的经验堆。

9.5 Triton 路线的调优边界

Triton 隐藏线程级细节，但 `BLOCK_SIZE`、`num_warps`、布局 and 访问顺序仍直接决定性能。块太小会产生许多 program 与重复开销，块太大可能造成寄存器溢出；warp 太少无法并行归约，太多又会争用资源。最佳配置随 shape、dtype 和 GPU 代际变化，因此应使用有限的候选集合做自动调优，并缓存结果。

检查生成 PTX/SASS 的目的不是追求手工汇编，而是验证高级意图：加载是否向量化，mask 是否产生昂贵控制流，局部数组是否落到 local memory，数学函数是否走预期近似，矩阵乘是否进入 Tensor Core。若生成代码不理想，先调整块级表达；只有编译器边界明确时才回退 CUDA。

自动调优也需要防止过拟合

调优集应覆盖真实形状分布，并在独立验证集上确认。只对一个尺寸搜索出的配置可能利用偶然对齐，在邻近尺寸严重退化。生产 dispatch 还要限制编译缓存大小和首次请求延迟。

9.6 Softmax 的完整检查

softmax 是很好的综合题：读取一行、求最大值、指数、求和、归一化并写回。数值上要减最大值；性能上要避免三个独立 kernel 和中间张量；布局上要让一行连续读取；资源上要让整行或合理分块进入片上；边界上要把 mask 位置视作负无穷；反向还要决定保存概率还是重算。

短行时固定 launch 成本明显，融合价值大；中等行适合单 program 完成归约；极长行可能超过寄存器或共享内存能力，需要分层归约。因而“一个 Triton softmax”在生产中往往是多条路径。课堂实现提供最小可理解核心，工程实现则要用 dispatch 包围这个核心。

提交前的六个问题

所有支持输入都正确吗？梯度与参考一致吗？最快路径覆盖了真实高频形状吗？退化形状有回退吗？首次编译与稳态成本都可接受吗？性能回归是否在 CI 可重复？六项都回答后，局部加速才成为可维护能力。

9.7 本章小结

从 PyTorch 到 Triton、CUDA 的每次下沉，都用更多工程责任换更细控制。生产级 kernel 必须同时拥有语义契约、正确性矩阵、可信 benchmark、可反证 profiler 结论、生成代码检查和明确回退。性能数字只是最终证据之一，不是全部交付物。

进一步说，优化记录还应保存失败方案。某个更大 tile 因寄存器溢出退化、某次融合因图断裂没有生效、某个近似函数造成精度越界，这些负结果能阻止团队反复走同一条路。把输入范围、硬件、编译器版本、观测指标和结论写进回归报告，kernel 知识才不会只留在作者记忆里。课程真正训练的能力，是用最少但充分的实验，把“看起来可能更快”变成可复现、可解释、可维护的工程事实，并让后来者可以重新验证全部结论。